



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,
SCIENZE FISICHE E SCIENZE DELLA TERRA

Bachelor's Degree in Computer Science L31

Curriculum: Data Analysis

**Towards Autonomous Industrial Mobility: An
AI Framework for Optimal Path Planning and
Resource-Efficient Navigation**

Supervisor:
Prof. Giovanni Merlino

Candidate:
Arian Shafagh

Academic Year:
2025/2026

Abstract

<abstract text>

Keywords: autonomous mobile robots, neuro-symbolic AI, deep reinforcement learning, energy-aware navigation, ROS 2, Nav2, simulation.

Contents

Abstract	1
Acknowledgements	10
1 Introduction	11
1.1 Motivation	11
1.2 Problem statement	11
1.3 Research questions	11
1.4 Contributions	12
1.5 Outline	12
2 Background and Related Work	13
2.1 Autonomous mobile robots in production logistics	14
2.2 Navigation: ROS 2 and Nav2	14
2.3 Decision making for mobile robots	14
2.3.1 Rule-based and heuristic dispatching	14
2.3.2 Deep reinforcement learning	14
2.3.3 Neuro-symbolic approaches	14
2.4 Safety in learned policies	14
2.5 Energy-aware navigation and battery management	14
2.6 Simulation and sim-to-real validation	14
2.7 Research gap	14

3	Problem Setting	15
3.1	The factory	15
3.2	The robot	18
3.3	What the robot can do	19
3.4	What counts as a good shift	20
3.5	Constraints	21
3.6	Why the problem is not trivial	21
3.7	Scenarios	22
4	System Design	25
4.1	Architecture	25
4.2	One description of the world	27
4.3	The factory model	27
4.4	The robot model	28
4.5	Executing an action	29
4.6	The decision service	30
4.7	Monitoring	30
4.8	Two simulators	31
5	Decision Models	32
5.1	The rule-based reference	32
5.2	The neuro-symbolic model	33
5.2.1	Hard rules	34
5.2.2	Priority tiers	35
5.2.3	Explanation	36
5.3	The neural scorer	36
5.3.1	Features	36
5.3.2	Learning a correction, not a value	37
5.3.3	Training	38
5.4	Bounding the correction	38

5.5	The reinforcement learning comparison	40
5.6	Ablations	41
6	Experimental Setup	43
6.1	Evaluation protocol	43
6.2	Keeping training and testing apart	44
6.3	Metrics	45
6.4	Statistics	46
6.5	Validation in the full simulation	46
6.6	The planner study	48
6.7	Software and hardware	48
7	Results	49
7.1	Overall comparison	49
7.2	Per-scenario behaviour	51
7.3	What each half contributes	52
7.4	Generalisation	53
7.5	Safety	54
7.6	Throughput and energy	55
7.7	Decision latency	56
7.8	Validation against the full simulation	56
7.9	Global path planner	58
7.10	Summary	59
8	Discussion	60
8.1	Answering the research questions	60
8.2	Why bounding the correction works	61
8.3	Why the rules matter more than the score suggests	63
8.4	Measurement decisions that changed the conclusions	63
8.5	Limitations	64

8.6	Threats to validity	66
9	Conclusion and Future Work	67
9.1	Conclusion	67
9.2	Future work	67
A	Configuration Parameters	69
B	Scenarios	70
C	Reproducing the Results	71

List of Figures

3.1	The production hall: three sections (A north-west, B north-east, C south-east), one delivery point and one charging station. Every corridor is at least 1.5 m wide, and the layout is deliberately asymmetric so that different places produce distinguishable sensor readings.	16
4.1	System architecture. Everything geometric derives from one file (top); the decision service is a separate process that exchanges plain data with the executor, and the dashboard only observes.	26
5.1	The neuro-symbolic decision pipeline. The symbolic layer removes actions and ranks what remains into urgency tiers; the neural scorer only ever chooses <i>within</i> the most urgent tier, and only by at most β score units. .	34
7.1	Score per policy and scenario (mean over 30 seeds, 95 % bootstrap CI). Hatched bars mark policies whose mean includes shifts with safety violations; the annotation gives how many of the 30 shifts were affected. . . .	51
7.2	Share of shifts containing at least one safety violation (battery below the reserve away from the charger, motor temperature at the limit, or an empty battery).	54
7.3	Delivered units against energy per delivered unit, averaged over all scenarios and seeds. Points towards the lower right deliver more for less energy.	55

7.4	Gazebo score against fast-simulator score for the same policy, scenario and seed. A point on the dashed line would mean perfect agreement.	57
-----	--	----

List of Tables

3.1	Shortest path through the maze between the points of interest, in metres. These are the true walkable distances, computed on the occupancy grid, not straight lines.	17
3.2	The three production sections at nominal rates. “Time to full” assumes an empty buffer and no interruption.	17
3.3	The twelve scenarios. “Seen” scenarios are available for training; “unseen” scenarios were written after training and are used only for evaluation. . .	23
5.1	Choosing the bound β : mean score over the six training scenarios with validation seeds 1000 to 1019. $\beta = \infty$ is the unbounded model, $\beta = 0$ switches the network off and decides on the analytic estimate alone. . . .	40
5.2	The policies compared in Chapter 7 and what each one isolates.	42
7.1	Results over all twelve scenarios (360 shifts per policy). Δ is the mean paired difference in score against the neuro-symbolic model; p is the paired Wilcoxon signed-rank test on identical seeds. “Unsafe shifts” counts shifts with at least one safety violation.	50
7.2	Score per scenario (mean over 30 seeds). * marks a difference from the neuro-symbolic model that is significant at $p < 0.05$ (paired Wilcoxon). Entries marked <i>unsafe</i> contain shifts with safety violations and are therefore not comparable on score alone.	52

7.3	Seen against unseen scenarios (180 shifts each). The bound costs nothing on the scenarios the network was trained on and gains substantially on those it was not.	53
7.4	Full sixty-minute shifts in Gazebo, scenario <code>low_battery_start</code> , seed 5000.	58

Acknowledgements

<optional; thank supervisors, reviewers, family. Keep it short and personal.>

Chapter 1

Introduction

1.1 Motivation

1.2 Problem statement

1.3 Research questions

RQ1. *<Does a neuro-symbolic policy outperform a deep reinforcement learning policy of equal information on throughput, energy and safety?>*

RQ2. *<What does each half contribute — do the rules alone, or the network alone, suffice?>*

RQ3. *<Do the policies still hold up in situations they were never trained on?>*

RQ4. *<Do results obtained in a fast abstract simulator transfer to full physics simulation with a real navigation stack?>*

1.4 Contributions

- A complete, reproducible ROS 2 system in which a single robot works an autonomous shift: factory model, energy/thermal/wear model, mission executor with recovery, decision service and a read-only monitor.
- A neuro-symbolic model whose hard rules can never be overruled, whose priority tiers are explicit, and whose learned ranker is *bounded*: the correction it may apply is limited to ± 1.0 score units.
- The observation that motivates that bound: without it, the learned ranker carried a habit from its training scenarios into an overloaded factory it had never seen and waited while production lines stopped (33.78 against 54.88 with the bound).
- An evaluation of 3240 simulated shifts with paired statistics and a held-out set of scenarios, validated by full Gazebo shifts in which the energy model agrees to $+0.2\%$.
- A planner study showing that a global planner selected on short benchmark tours was the worse choice over full shifts (6 of 17 shifts faulty against 0 of 5).

1.5 Outline

Chapter 2

Background and Related Work

2.1 Autonomous mobile robots in production logistics

2.2 Navigation: ROS 2 and Nav2

2.3 Decision making for mobile robots

2.3.1 Rule-based and heuristic dispatching

2.3.2 Deep reinforcement learning

2.3.3 Neuro-symbolic approaches

2.4 Safety in learned policies

2.5 Energy-aware navigation and battery management

2.6 Simulation and sim-to-real validation

2.7 Research gap

Chapter 3

Problem Setting

This chapter describes the world the robot works in, the choices it has, and the yardstick by which its behaviour is judged. Everything stated here is a property of the problem, fixed before any solution is designed; the decision models that answer it follow in Chapter 5. The chapter closes by showing, with the numbers of the setting itself, why the question *what should the robot do next?* has no obvious answer.

3.1 The factory

The setting is a single production hall of 14.5m by 10m, laid out as a maze of walls, machines and corridors (Figure 3.1). Three production sections, named A, B and C, stand in different regions of the hall. Each of them manufactures finished units and places them in a local output buffer. A single delivery point in the south of the hall accepts finished units, and a single charging station in the south-west corner is where the robot starts its shift and the only place it can recharge.



Figure 3.1: The production hall: three sections (A north-west, B north-east, C south-east), one delivery point and one charging station. Every corridor is at least 1.5 m wide, and the layout is deliberately asymmetric so that different places produce distinguishable sensor readings.

The geometry is not decorative. Two properties of it shape the decision problem. First, the distances between the points of interest differ by almost a factor of three (Table 3.1), so the order in which sections are served changes how far the robot drives and therefore how much energy it spends. Second, several pairs of points are connected by more than one route, and the shortest route is not always the one a planner finds — a fact that becomes important in Section 7.9.

Table 3.1: Shortest path through the maze between the points of interest, in metres. These are the true walkable distances, computed on the occupancy grid, not straight lines.

	A	B	C	delivery	charger
A	—	18.28	15.56	10.01	9.04
B	18.28	—	9.24	10.26	15.18
C	15.56	9.24	—	6.30	11.27
delivery	10.01	10.26	6.30	—	5.10
charger	9.04	15.18	11.27	5.10	—

The three sections are deliberately unequal (Table 3.2). Section B is the fast line: it produces one unit every two minutes, and its buffer of ten units is full after twenty minutes of neglect. Section A is half as fast with a slightly larger reserve of time. Section C produces only one unit every twelve minutes, but its units are almost four times as heavy as B’s and its buffer holds just three of them, so it can be ignored for a while — but not forgotten. Production is not perfectly regular: cycle times are drawn from a Gamma distribution with a coefficient of variation of 0.15, so buffers do not fill on a predictable schedule.

Table 3.2: The three production sections at nominal rates. “Time to full” assumes an empty buffer and no interruption.

Section	Rate (units/h)	Buffer (units)	Unit mass (kg)	Full buffer (kg)	Time to full
A	20	8	0.5	4.0	24 min
B	30	10	0.4	4.0	20 min
C	5	3	1.5	4.5	36 min

What turns this into a problem rather than a chore is the consequence of a full buffer. A section whose buffer is full cannot put the next finished unit anywhere, so the line stops: it enters the state BLOCKED, and everything it would have produced while blocked is lost for good. Lost production cannot be recovered later by working harder, which is precisely why the robot’s decisions matter. Two further states model the wear of the machines themselves. A machine’s health decreases slowly with every unit it makes;

below 60% health it runs at a reduced rate (DEGRADED), and it can break down entirely (FAULT), which stops production for a repair of about twenty minutes. Faults are more likely the less healthy a machine is.

3.2 The robot

The transport robot is a small differential-drive platform of the TurtleBot class: 4.3 kg, a two-dimensional lidar for localisation, and a single-board computer. Its measured average speed through this maze, turns included, is 0.39 m/s. It carries at most 5 kg at a time, and loading or unloading takes 30 s of standing still.

Four physical properties constrain what it can do, and all four are modelled explicitly rather than assumed away (the full parameter set is in Appendix A):

Energy.

The robot draws 6 W whenever it is switched on, plus 0.005 W h/m while driving empty and a further 0.0016 W h/m for every kilogram it carries. Its battery holds 22 W h, which is roughly 1.7 h of continuous driving — enough for a shift only if the robot does not drive needlessly.

Charging.

The charging station delivers 25 W, so a full recharge takes about an hour. Charging is therefore not a pause but a decision with a real cost in lost working time, which is why the robot can choose *how far* to charge rather than only whether to.

Heat.

The motors warm up under load and cool down when idle, reaching a steady state of about 55 °C at continuous full load in a 22 °C hall. Above 55 °C wear accelerates; at 70 °C the robot must stop and cool down. A heavy payload heats it faster.

Wear.

The drivetrain's condition degrades with energy drawn and with time spent hot. A

worn drive is not merely old: at 0 % condition it draws 30 % more energy for the same journey, and below 30 % the robot is due for maintenance and may no longer be given work.

These properties interact. Carrying more units per trip saves journeys but costs energy per metre and heats the motors faster; driving with a hot motor costs extra energy through reduced efficiency; and every watt-hour drawn wears the drive a little further, which in turn makes future journeys more expensive.

3.3 What the robot can do

The robot's behaviour is expressed as a sequence of complete actions rather than as continuous motion. One action is one decision, and the next decision is only taken when the current action has finished. Four kinds of action exist:

PICKUP: <A|B|C>

Drive to the named section and load as many waiting units as still fit within the payload limit, emptying that much of its buffer.

DELIVER

Drive to the delivery point and unload everything carried. Only delivered units count as produced value.

CHARGE: <40|70|100>

Drive to the charging station and charge up to the chosen level. Offering three levels makes the trade-off explicit: a quick top-up returns the robot to work sooner, a full charge buys independence for longer.

WAIT: <s>

Stand still for sixty seconds. At the charging station this is free and tops up the battery; anywhere else it merely spends idle power.

Driving itself is not part of the decision problem. A navigation stack plans and follows the route, recovers from obstacles, and reports success or failure (Section 4.5); the decision layer chooses *where* to go, never *how* to get there.

3.4 What counts as a good shift

A shift lasts one hour of factory time. Everything the robot does during it is judged by a single number, so that different behaviours can be compared without argument:

$$J = n_{\text{delivered}} - n_{\text{lost}} - 0.5 E_{\text{battery}} - 20 n_{\text{violations}} \quad (3.1)$$

where $n_{\text{delivered}}$ is the number of units brought to the delivery point, n_{lost} the production lost while lines stood blocked, E_{battery} the energy drawn from the battery in watt-hours, and $n_{\text{violations}}$ the number of safety violations (defined in Section 3.5).

The weights encode a position that is worth stating plainly. A delivered unit and a lost unit weigh the same, because a unit that never got made is as costly as one that never arrived. Energy is deliberately cheap: at 0.5 per watt-hour, and with a measured journey between two points of the hall costing roughly 0.18 W h to 0.25 W h, no reasonable policy can improve its score by refusing to work — energy is a tie-breaker between journeys, not a reason to stay at home. A safety violation, in contrast, costs twenty units, far more than any shift’s worth of careful energy saving, because a stranded or overheated robot is not a slightly worse outcome but a different kind of outcome: it requires a human being to walk into the hall.

Two things this objective deliberately does not charge for. Time spent idle is free, since the shift has a fixed length and an idle robot in a calm factory harms nothing. And only energy drawn *from the battery* is counted, not the energy the charging station supplies while the robot is docked. The second choice is a simplification whose consequences are revisited honestly in Section 8.5.

3.5 Constraints

Four conditions must hold at all times, and breaking any of them counts as a safety violation in Equation (3.1):

C1. Battery reserve. At least 15% of the pack must remain after the robot has returned to the charging station. The reserve is not a threshold on the current battery level but on the level *after the journey home*, which means the constraint depends on where the robot is and what it is carrying.

C2. Temperature. Motor temperature must stay below 70°C.

C3. Payload. The robot may never plan to carry more than 5 kg.

C4. Maintenance. Below 30% drive condition the robot may only charge or wait; it is no longer fit for transport work.

Constraint C1 is the interesting one, because it cannot be checked by looking at the battery gauge. Answering *may I fetch from section B?* requires simulating the whole journey — there, loaded, and home again — through the robot’s own energy, thermal and wear models. The same question asked with a hot motor, a heavy load or a worn drive has a different answer.

3.6 Why the problem is not trivial

The setting was designed so that no simple rule is obviously right. Three pieces of arithmetic show why.

The robot is not comfortably faster than the factory. At nominal rates the three lines together produce 55 units per hour with a combined mass of 29.5 kg. With a payload limit of 5 kg, at least six loaded deliveries per hour are needed just to keep pace, and each of them costs travel time to the section, loading, travel to the delivery point

and unloading. The robot has enough capacity for the nominal factory, but not much to spare — and the scenarios in Section 3.7 include days where the demand exceeds what one robot can carry, so that some loss is unavoidable and the only question is how little.

Urgency and cost point in different directions. Section B fills fastest and is therefore usually the most urgent, but it is also the section farthest from the charging station (15.18 m) and from section A (18.28 m). Section C fills slowest, but sits closest to the delivery point (6.30 m) and carries the heaviest units, so serving it is cheap per journey and expensive per kilogram. A policy that always serves the most urgent line drives further than one that batches nearby work; a policy that always minimises distance eventually lets a line block.

Charging competes with everything else. Recharging from empty takes about an hour at 25 W — a whole shift. A robot that charges too early gives up working time it cannot get back; one that charges too late violates C1, and the penalty for that is worth twenty delivered units. Because the reserve must hold *after* the journey home, the moment at which charging becomes unavoidable depends on where the robot happens to be standing.

None of these tensions can be resolved once and written down as a fixed order of visits, because the state that decides them — buffer fills, machine health, battery, temperature, condition — changes continuously and partly at random.

3.7 Scenarios

A single factory configuration would measure very little, so the problem is instantiated in twelve scenarios (Table 3.3). Each one changes the parameters of Section 3.1 or Section 3.2 — production rates, buffer sizes, unit masses, machine health, ambient temperature, battery capacity or the robot’s initial state — while the objective and the constraints stay the same.

Six of the scenarios were written before the decision models were trained, and both learned models in Chapter 5 use exactly those six. The other six were written afterwards, when the models already existed, and are never used for training or for selecting a model. They therefore measure something the first six cannot: whether a policy still behaves sensibly in a factory it has never seen. This separation is what makes Section 7.4 a measurement rather than a claim, and it is maintained down to the random seeds (Section 6.2).

Table 3.3: The twelve scenarios. “Seen” scenarios are available for training; “unseen” scenarios were written after training and are used only for evaluation.

Scenario	Split	What it stresses
balanced	seen	Nominal rates, rare faults, healthy fully charged robot.
high_demand	seen	All lines at twice the nominal rate; robot utilisation about 0.85.
one_hot_section	seen	Section B at double speed, blocking within ten minutes if neglected.
fault_burst	seen	Worn machines failing about once per hour with long repairs.
low_battery_start	seen	Robot starts at 30 % battery with buffers already half full.
worn_robot	seen	Drive condition 45 %: more energy per metre, close to the maintenance limit.
heavy_load	unseen	Double production, heavier parts, half-charged robot: slightly beyond one robot’s capacity.
heavy_parts	unseen	Units about twice as heavy, so fewer fit per trip.
small_buffers	unseen	Lean production: buffers hold only a few units and block within minutes.
section_breakdown	unseen	Section B’s machine failing (health 35 %) while A and C run normally.
hot_factory	unseen	38 °C in the hall and poor cooling: long heavy trips risk the temperature limit.
aged_battery	unseen	Pack down to 12 W h (55 % of new) and a slower 15 W charger.

With the world, the actions, the objective and the constraints fixed, the question of this thesis can be stated exactly: *which policy, choosing one action at a time from*

the state of the factory and the robot, maximises Equation (3.1) without ever breaking C1–C4 — including in factories it was never designed or trained for?

Chapter 4

System Design

The decision models of Chapter 5 are only as trustworthy as the system that executes them and the simulator that measures them. This chapter describes that system: how the world is generated from a single description, how the factory and the robot are modelled, how an action is turned into motion and what happens when the motion fails, and how a decision model is attached to the running robot without becoming part of it.

4.1 Architecture

The system is built as a set of ROS 2 nodes around a Gazebo simulation, with the decision model as a separate process (Figure 4.1). Four elements matter for the rest of this thesis:

- a **generator** that turns one layout description into the simulated world, the navigation map, the named poses and the table of maze distances;
- a **factory node** that runs the production model of the three sections and publishes their live state;
- a **mission executor** that turns one action into navigation goals, dwell times and a report, and that recovers when navigation fails;

- a **decision service** that answers the executor’s question *what next?* and can be stopped, replaced or upgraded without touching the robot.

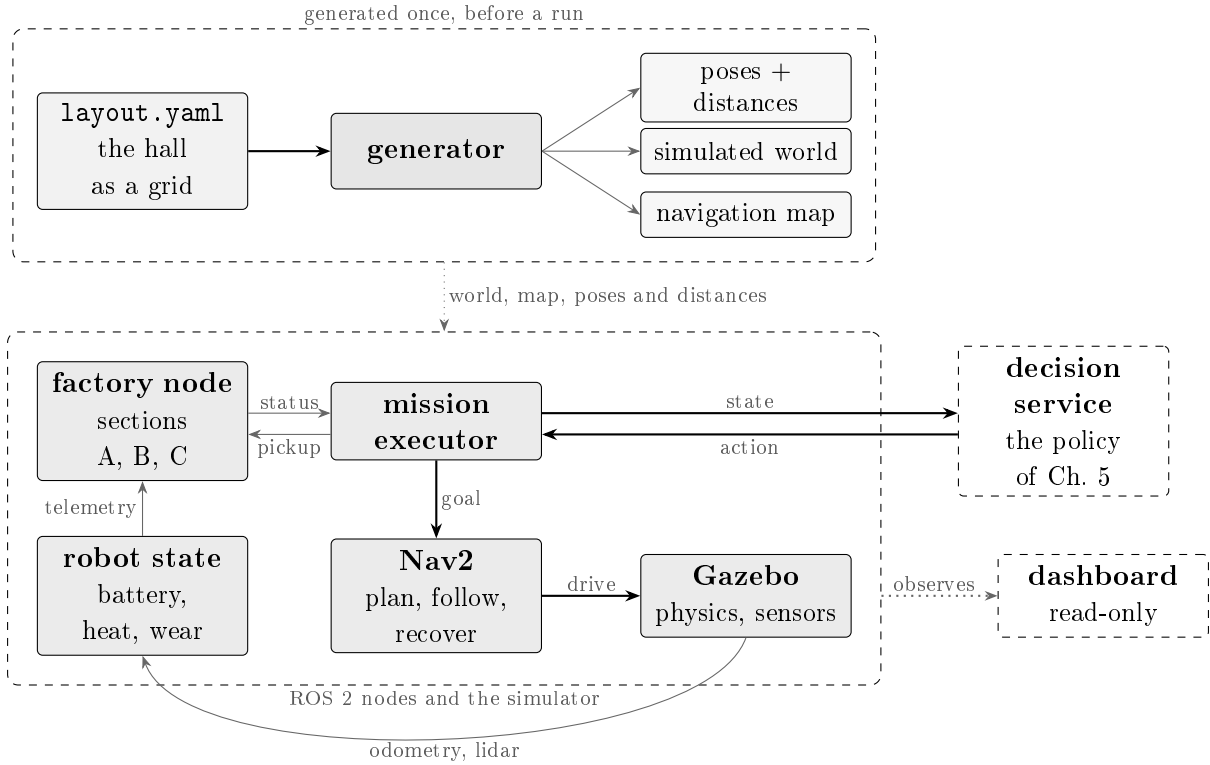


Figure 4.1: System architecture. Everything geometric derives from one file (top); the decision service is a separate process that exchanges plain data with the executor, and the dashboard only observes.

One principle runs through the design: **the decision layer never sees ROS**. A policy receives plain data and returns an action, which is what allows the identical policy object to run inside the fast simulator for thousands of training shifts and inside the live service on the robot. If the two had separate implementations, every result in Chapter 7 would depend on their agreeing, and that agreement would have to be proved rather than assumed.

4.2 One description of the world

The hall is described once, as an ASCII grid in which each character is a half-metre cell: wall, machine, pickup zone of a section, delivery point, charging station or free floor. A generator turns that single file into four artefacts: the Gazebo world, the occupancy map used for navigation, the named poses the robot drives to, and a matrix of the shortest maze path between every pair of those poses.

The alternative — maintaining a simulated world and a navigation map separately — is a known trap: the two drift apart, and the robot then plans in a hall that differs from the one it drives in. Deriving both from one description makes that class of error impossible, and it has a second benefit for this thesis: the path matrix used by the fast simulator and by the symbolic rules is generated from the same geometry the robot actually drives, so a distance in Table 3.1 is not an approximation of the simulated world but a property of it.

A lesson from the first layout. The first version of the hall contained a 0.5 m gap between two wall segments. Nav2 treats only about 0.22 m around an obstacle as lethal, so its planner happily routed through the gap, while the controller could not fit the robot through it. The result was a robot that planned 10.47 m from B to C, and then drove 24.4 m after re-routing. The layout was widened, but the more useful outcome is the check that was added to the generator: it compares the shortest path through comfortable space with the shortest path through any space wider than 0.25 m, and refuses to generate a world if the second is more than 5 % shorter for any pair of points — which is exactly the signature of a gap the robot cannot use. Fed the old layout, the check flags precisely the three pairs Nav2 had got wrong.

4.3 The factory model

Each section is modelled as a machine with an output buffer. Cycle times are drawn from a Gamma distribution, so units appear irregularly around the nominal rate; finished

units accumulate in the buffer; and when the buffer is full the line stops and everything it would have produced is counted as lost. Machine health decreases with every unit produced, reduces the production rate below 60% health, and raises the hazard of a breakdown, which stops the line for a repair of about twenty minutes. The model is pure Python with a seeded random generator per section, and it is used unchanged by the fast simulator and by the ROS node, so a scenario behaves the same in both.

The three sections run inside a *single* ROS node rather than one node each. This began as a performance fix — three Python nodes on the simulation clock, each waking on every clock message, starved the navigation stack during start-up — and the same investigation led to the physics step being set to 4ms. It is a small detail, but it is the reason the system can run a full shift reliably on a laptop, which is what made the twelve Gazebo validation shifts possible.

4.4 The robot model

The robot’s energy, temperature and wear are modelled analytically rather than measured from Gazebo, for two reasons: the same model must run thousands of times per second during training, and both simulators must agree on what an action costs. Energy is idle power plus a distance term plus a payload term, corrected for motor temperature and drive condition; temperature integrates heating under load against cooling; condition degrades with energy drawn and with time spent above the warning temperature (Section 3.2).

Because every result in this thesis is denominated in these units, the model is checked rather than trusted. A test asserts that its parameters imply physically plausible magnitudes — an energy consumption between 5 W h/km to 40 W h/km, a continuous driving time between one and four hours, a full charge between thirty minutes and two hours. That test caught the first calibration error in this work: a drive energy of 0.012 W h/m, which reads plausibly but implies 16.8 W of motor power for a 4.3 kg robot and a runtime under an hour. The corrected value, 0.005 W h/m, is the one used throughout.

4.5 Executing an action

The mission executor is the only component that touches both worlds: it receives an action from the decision service and produces motion, dwell and a report. For a pickup it sends the section's pose as a navigation goal, waits for arrival, stands still for the loading time on the simulation clock, asks the section to hand over as many units as still fit, and records what the action cost — duration, distance, energy, battery — next to what the model had predicted it would cost. That pairing of predicted and measured values is what later allows the fast simulator to be validated action by action.

Navigation failure is normal, not exceptional. Over a full shift a robot in a maze will fail to reach a goal: a costmap holds a stale obstacle, the robot ends up wedged against a wall, a planner refuses. The executor therefore treats failure as part of the job and escalates in fixed steps: retry, clear both costmaps and retry, back up 0.3m and retry, and finally abandon the goal and return to the charging station as a safe place. Only if the charger is unreachable as well does the robot stop and end the mission as *navigation stuck*, which is the one outcome that requires a person. Each drive also carries a timeout of three times its predicted duration, so a robot that oscillates cannot hang the shift. Section 7.9 contains a shift in which this ladder ran to its end.

Arrival is verified, not believed. Nav2 can report success without the robot being at the goal: when a goal pose is blocked, its planner plans to the nearest free position within its tolerance and the controller duly arrives *there*. Every reported success is therefore checked against the robot's localised position, and a deviation beyond 0.35m counts as a failure. Without this check a blocked pickup point would silently become a successful pickup of nothing, and the resulting data would be wrong in a way that no later analysis could detect.

4.6 The decision service

The decision model runs as its own process and answers over HTTP: the executor sends the current state and the legal actions, the service returns the chosen action together with its scores and its explanation. The separation buys three things. The model can be changed — or swapped for another model entirely — without restarting the robot; “the AI is unavailable” becomes a state that can be produced deliberately, by stopping the process; and the decision latency reported in Section 7.7 is measured on the same path the robot uses, not in a test harness.

If the service does not answer, the executor falls back to the rule-based policy of Section 5.1 and continues the shift, recording for each decision whether it came from the model or from the fallback. A robot that stops working because a web service is down would be of no industrial interest; this way the failure degrades the quality of the decisions rather than the availability of the robot.

4.7 Monitoring

A web dashboard shows the live state: the three sections with their buffers, production and health, the robot with its battery, temperature, payload and position on a map of the hall, the shift totals, and the stream of decisions with their explanations. It is deliberately read-only — there is no control on the page, and no route that accepts a command. The robot is autonomous by construction, and the operator’s only intervention is the physical emergency stop, not a button in a browser.

The map on the page is drawn from the same layout description as the simulated world (Section 4.2), so what the operator sees cannot drift from where the robot actually is.

4.8 Two simulators

The system exists twice. The full simulation — Gazebo physics, Nav2, localisation, the executor — is faithful but runs in real time: one shift takes an hour. The fast simulator replaces driving with the path matrix and the robot model, advances the same factory model in one-second steps, and completes a shift in milliseconds. Training the neural scorer and evaluating nine policies over 3240 shifts is possible only in the second; claiming that those results describe the first requires evidence.

That evidence is produced by replaying a completed Gazebo mission action by action in the fast simulator and comparing the predicted values against the measured ones. For the reference mission the totals agree to 0.6% in energy, with the worst single action differing by 10% in duration. The shift-level comparison of Section 7.8 extends this from a single scripted mission to complete autonomous shifts under both decision models.

Chapter 5

Decision Models

The system of Chapter 4 can execute any action; this chapter is about choosing which one. Four answers are developed and compared: a rule-based reference policy, the neuro-symbolic model that is this thesis’s contribution, a deep reinforcement learning policy, and a family of ablations that isolate what each part of the neuro-symbolic model actually contributes.

All of them implement the same interface. A policy receives the state of the factory and the robot, together with the list of currently legal actions, and returns one action, a set of scores and a human-readable explanation of why. Nothing in a policy touches ROS, which is what allows the identical object to run inside the fast simulator during training and inside the decision service on the robot (Section 4.6).

5.1 The rule-based reference

The reference policy exists so that the learned models have something honest to beat. It is written from what an operator would say about the job, in five sentences:

1. Keep the robot able to get home: charge when the battery would not cover the trip plus the journey to the charger plus the reserve.

2. Never let a line stand still: serve BLOCKED sections first, then whichever will block soonest.
3. Do not drive half empty: top the load up at another section if it is close and has units.
4. Deliver what you carry when the robot is full or nothing else is worth collecting.
5. Wait at the charger, never in an aisle, because waiting anywhere else only spends energy.

A section counts as urgent if it will block within 180s, and a buffer below 35 % is not worth a dedicated trip. The policy is roughly a hundred lines long, entirely explainable, and — as Section 7.1 shows — far from useless: it scores 41.66 against the neuro-symbolic model’s 53.81, and it is the most energy-efficient policy in the comparison. It is also the fallback the robot runs when the decision service is unavailable (Section 4.6), so the same code has to be trustworthy in production, not only in the evaluation.

5.2 The neuro-symbolic model

The neuro-symbolic model is built from the observation that the decision has two different kinds of content. Some of it is arithmetic and physics: whether the robot can complete a journey without stranding itself, whether a line is already losing production, how much energy an action costs. That part is known exactly, it can be computed from the robot’s own models, and getting it wrong is unacceptable rather than merely suboptimal. The rest is judgement: whether it is worth driving past a half-full buffer to reach a fuller one, whether to collect a second load before delivering, whether a quick top-up now saves a longer charge later. That part is not exactly computable, but it can be learned.

The model separates the two. A symbolic layer decides what is *allowed* and what is *urgent*; a neural scorer chooses among the candidates that survive. Figure 5.1 shows the pipeline.

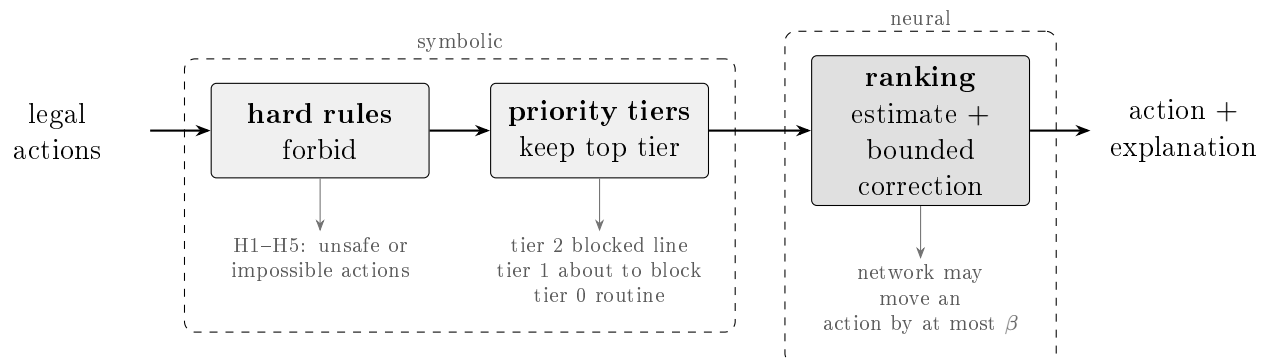


Figure 5.1: The neuro-symbolic decision pipeline. The symbolic layer removes actions and ranks what remains into urgency tiers; the neural scorer only ever chooses *within* the most urgent tier, and only by at most β score units.

5.2.1 Hard rules

Hard rules forbid. They encode the constraints of Section 3.5 and no score from any network can overrule them — this is the guarantee that a purely learned policy cannot give.

H1 — Maintenance.

Below 30% drive condition, pickups and deliveries are forbidden; only charging and waiting remain.

H2 — Payload.

A pickup is forbidden if nothing at that section still fits within the remaining payload capacity.

H3 — Reserve.

An action is forbidden if the battery after completing it *and returning to the charger* would fall below the reserve plus a safety margin of 2%. This is not a threshold on the gauge: the whole route — outward leg, dwell, journey home — is simulated with the robot’s own energy, thermal and wear models (Section 4.4), so the answer depends on the payload, the motor temperature and the condition of the drive.

H4 — Heat.

An action is forbidden if the simulated peak motor temperature along that route would reach the limit of 70°C.

H5 — Pointless waiting.

Away from the charger, if every pickup and delivery is already forbidden and charging is possible, waiting is forbidden too.

H1–H4 follow directly from the constraints. H5 does not: it was added after watching the robot work a real shift, and the reason is worth recording. Charging becomes *urgent* (tier 2 below) slightly below the battery level at which the pickups are already refused by H3. In the gap between those two thresholds, waiting and charging looked equally good to the ranker, and the robot spent about 1 % of its pack waiting at a section before driving to the charger anyway. The rule closes the gap by stating what is obviously true and was nowhere written down: if the robot cannot collect or deliver anything until it charges, waiting cannot improve the situation. Adding it improved the score on the training scenarios from 58.70 to 58.81 and, on the full evaluation, `low_battery_start` from 55.14 to 55.57.

5.2.2 Priority tiers

The surviving actions are then ranked into three tiers, and only the most urgent tier is passed on:

Tier 2 — unblock a stopped line.

A pickup at a BLOCKED section, or charging when the battery is at the reserve.

Tier 1 — prevent a line from stopping.

A pickup at a section that will fill within 180s or within the travel time to it, whichever is longer; or a delivery when the robot is full and can collect nothing else.

Tier 0 — routine.

Everything else.

The tiers express a priority that should never be traded away for a small gain in energy: a line that is already losing production outranks one that is merely getting full, which outranks a convenient errand. Because only the top tier reaches the scorer, the network can never choose a routine action while a line stands still — it can only decide *which* of the equally urgent actions to take.

5.2.3 Explanation

Every verdict carries its reasons, so every decision explains itself. A real decision from a Gazebo shift reads:

```
decision: WAIT:60 -- [routine] waiting away from the charger only spends
energy | PICKUP:B refused: battery would be 16 % back at the charger,
below the 17 % reserve; DELIVER refused: battery would be 17 % back at
the charger, below the 17 % reserve; neural score +0.95, chosen over CHARGE:40
by +0.16
```

The explanation names the tier, the reason the chosen action is in it, which alternatives were forbidden and why, and by how much the network preferred this action to the runner-up. This is not a post-hoc rationalisation of a black-box output: it is the actual computation, written out. It is also how the H5 rule was found — the string above is the trace that showed the robot preferring to wait while it was in fact stuck until it charged.

5.3 The neural scorer

5.3.1 Features

The network scores *one candidate action at a time*: it receives a vector of 27 features describing that action in the current state, and returns a single number. Scoring actions rather than states is what keeps the network small and portable — it learns “is this action worth it, given what it costs and what it saves”, not “what should be done in this exact factory”.

The features cover four groups: what kind of action it is; the robot’s state (battery, temperature, condition, payload, cargo, time left in the shift); what the action costs and leaves behind (distance, energy as a percentage of the pack, battery afterwards, battery after also returning home); and what it achieves (the target section’s fill, units and mass gained, time to full, rate, health, whether it is blocked or faulted, plus the worst other section’s fill and time to full, and whether any other line is blocked). Charging actions additionally carry their target level and the gain it would bring.

5.3.2 Learning a correction, not a value

The network does not predict an action’s value. It predicts a *correction* to an analytic estimate:

$$\text{score}(a) = \underbrace{\hat{v}(a)}_{\text{symbolic estimate}} + \underbrace{c_{\theta}(a)}_{\text{learned correction}} \quad (5.1)$$

where the symbolic estimate is the obvious part of the value, computed directly from the models: the units the action moves, valued with the objective of Equation (3.1), minus the energy the journey and its dwell time cost. What the estimate cannot know is the future — which line blocks next, whether a fuller load would have been better, whether this trip makes a charge unavoidable later — and that is exactly what the correction has to supply.

This split matters for a reason that only became visible in the evaluation (Section 5.4). The estimate is arithmetic on the current state, so it stays correct in any factory, including one with twice the production rate. The network is a function fitted on a distribution of states, so it is reliable near the situations it was trained on and unreliable away from them. Starting from the estimate means the model degrades towards sensible arithmetic rather than towards noise.

5.3.3 Training

Training data are collected by look-ahead in the fast simulator. At each decision point of a simulated shift, every candidate action is taken in a copy of the simulator, the resulting future is played out for 900 s with a reference policy, and the discounted return is recorded as that action’s value. Because production and faults are random, three futures are averaged per action. Exploration is added by choosing a random action at 25 % of decision points, so the data are not confined to what the reference policy would have done. Collection runs over the six training scenarios with seeds from 12 345 upwards, which never overlap the evaluation seeds (Section 6.2).

The decisive design choice is the loss. The look-ahead value of an action is noisy, and the candidates at one decision point often differ by far less than that noise, so regressing values spends the network’s capacity on modelling randomness. What the model actually has to get right is *which candidate is best*. The scorer is therefore trained with a softmax cross-entropy over the candidates of each decision point, with the look-ahead’s best action as the target, plus a small value-regression term that keeps the scores on a meaningful scale so they remain readable in the explanation strings.

Training is repeated for three rounds of policy iteration: the first round looks ahead with the symbolic policy, and each later round looks ahead with the model trained in the previous one. The round with the best validation score is kept. In the final training run, round 1 was the best (validation score 59.68, against 58.05 and 56.10 for rounds 2 and 3), so policy iteration did not pay off here — a negative result worth reporting rather than hiding. The kept model picks the look-ahead’s best action in 35 % of validation decision points, which sounds low until one notices that many decision points have several near-equivalent candidates; what matters is the shift score, not the agreement rate.

5.4 Bounding the correction

The model as described so far — rules, tiers, and a network correcting an analytic estimate — is the version that was first evaluated. It was worse than its own symbolic

layer on scenarios it had never seen (43.33 against 45.66, Table 7.3), and the gap came almost entirely from one scenario, `heavy_load`, where it scores 33.78 against the symbolic layer’s 53.21.¹

The decision trace explained why. In a factory producing beyond one robot’s capacity, the model repeatedly chose `WAIT` while carrying cargo, up to eight times in a row, while all three lines stood `BLOCKED` and lost production. Its own explanation string said, correctly, that “waiting away from the charger only spends energy”: the symbolic estimate ranked waiting last, and the learned correction — around +20 on every candidate, with differences of a few units — simply outvoted it.

The cause is not a bug but a distribution shift. In the six training scenarios, production is roughly half as fast, and waiting to accumulate a fuller load genuinely pays. The network learned that habit and applied it to an overload it had never seen, where the same habit is ruinous.

The remedy follows from Equation (5.1). The correction is centred on the mean of the candidates at that decision point — only differences between candidates affect the ranking — and then clipped:

$$\text{score}(a) = \hat{v}(a) + \text{clip}(c_\theta(a) - \bar{c}_\theta, -\beta, +\beta) \quad (5.2)$$

With a bound β , the network can reorder candidates whose analytic estimates lie within β of each other — the close calls, where judgement is what is needed — and can never overturn a difference the arithmetic considers clear. It constrains the network’s *influence*, which is a different thing from the action masking and shielding of Section 2.4: those constrain which actions are *legal*.

The bound was selected on the six training scenarios with validation seeds 1000 to 1019, never on the unseen scenarios or the evaluation seeds (Table 5.1).

¹These are the numbers of the final configuration, in which both variants also carry the H5 rule of Section 5.2.1. When the failure was first observed, before H5 existed, the same comparison read 28.5 against 53.21.

Table 5.1: Choosing the bound β : mean score over the six training scenarios with validation seeds 1000 to 1019. $\beta = \infty$ is the unbounded model, $\beta = 0$ switches the network off and decides on the analytic estimate alone.

β	∞	20	10	5	2	1	0
Mean score	57.75	57.75	58.06	58.15	58.15	58.70	57.41

Two things are visible in that table. The network is worth having: switching it off entirely ($\beta = 0$) is worse than every bounded setting. And its useful influence is small: the best value is $\beta = 1.0$, one point on a scale where a delivered unit is worth one point. The chosen model may therefore move an action by at most the value of a single delivered unit relative to the others — enough to break a tie, not enough to argue with the arithmetic.

What this costs and buys is measured in Section 7.3: nothing on the training scenarios ($p = 0.058$), 3.87 points on the unseen ones ($p < 0.001$), and 54.88 against 33.78 on `heavy_load`.

5.5 The reinforcement learning comparison

The learned comparison is Maskable PPO, trained on the same environment, the same observation content and the same objective, so that the comparison is about *where knowledge is placed*, not about who gets more information.

The agent sees the state as one vector (robot state plus per-section features) and chooses from the same discrete action set. Action masks mark the actions that are currently legal, which is the standard way of expressing hard constraints in reinforcement learning, and the agent samples only among them. Training runs for 300 000 steps across four parallel environments ($n_{\text{steps}} = 512$, batch size 256, learning rate 3×10^{-4} , $\gamma = 0.995$, $\lambda = 0.95$, entropy coefficient 0.01, two hidden layers of 64 units — the same size as the neural scorer), and takes about ten minutes on the evaluation machine.

One detail of the training setup had to be corrected before the comparison was fair. Each parallel environment was originally bound to one fixed scenario, so with four

environments the agent only ever saw four of the six training scenarios. The environment now draws a scenario at random at every reset, so all six are seen regardless of how many environments run. Both the corrected model and the original one are reported (`ppo` and `ppo_wp6`), because the difference between them is itself informative: it is small, and the safety behaviour does not change (Section 7.5).

A second variant, `ppo_unmasked`, is trained without the action mask. Illegal choices are executed anyway — they simply achieve nothing — and cost a small penalty, so the agent has to learn the rules from experience. It answers the question of how much of a masked agent’s competence comes from the mask rather than from learning.

5.6 Ablations

Table 5.2 lists every policy in the evaluation and the question it exists to answer. Each removes exactly one thing, so that a difference in Chapter 7 can be attributed.

Table 5.2: The policies compared in Chapter 7 and what each one isolates.

Policy	What it is, and the question it answers
<code>ns</code>	The full model: hard rules, priority tiers, analytic estimate, bounded neural correction.
<code>ns_unbounded</code>	The same model with $\beta = \infty$. <i>Does bounding the network matter?</i>
<code>ns_estimate_only</code>	The same model with $\beta = 0$. <i>Does the network contribute anything at all?</i>
<code>ns_symbolic_only</code>	Rules and tiers, then the cheapest action per unit moved; no network, no estimate. <i>How far do the rules alone get?</i>
<code>ns_neural_only</code>	The network ranking every legal action, with no rules and no tiers. <i>What did the network really learn — including about safety?</i>
<code>rule</code>	The reference policy of Section 5.1. <i>Is any of this better than common sense?</i>
<code>ppo</code>	Maskable PPO trained on all six training scenarios. <i>Rules versus learning, with equal information.</i>
<code>ppo_unmasked</code>	PPO without the action mask. <i>How much of PPO’s competence is the mask?</i>
<code>ppo_wp6</code>	The earlier PPO, trained on four of the six scenarios. <i>Did the training-coverage fix change the comparison?</i>

Chapter 6

Experimental Setup

This chapter states how the policies of Chapter 5 were measured, in enough detail that a reader who doubts a number in Chapter 7 can find out exactly how it was produced. Three questions decide whether such a comparison means anything: whether the policies were given the same problems, whether they were tested on situations they had not been tuned for, and whether the differences that are reported are larger than the noise. Each is answered below, followed by the protocol for the full-simulation validation and the software and hardware used.

6.1 Evaluation protocol

The main experiment runs every policy on every scenario for thirty different random seeds: $9 \text{ policies} \times 12 \text{ scenarios} \times 30 \text{ seeds} = 3240 \text{ shifts}$, each one simulated hour of factory time.

A seed fixes everything random in a shift: the production cycle times of the three sections, the moments at which machines fail and how long repairs take. The same seed therefore produces exactly the same factory for every policy, so the comparison is *paired*: for any two policies there are thirty matched pairs of shifts per scenario that differ only in the decisions taken. This matters because the variation between seeds is far larger than the variation between policies — scores within a single scenario routinely differ

by more between two seeds than the policies differ from each other on average — and pairing removes that variation from the comparison instead of leaving it as noise.

The fast simulator makes this affordable: the full experiment takes 78s on ten worker processes, which is what allows thirty seeds rather than three, and nine policies rather than two.

6.2 Keeping training and testing apart

A comparison of learned policies is only meaningful if they are measured on something they were not tuned for. Two separations are enforced.

Scenarios. Six scenarios — `balanced`, `high_demand`, `one_hot_section`, `fault_burst`, `low_battery_start` and `worn_robot` — are declared as the training set, and both learned models use exactly those six: the neural scorer collects its look-ahead data from them, and PPO trains on them. The other six (Table 3.3) were written after training, when both models already existed, and are used for evaluation only. They are referred to as *unseen* throughout, and results on them are reported separately (Section 7.4) rather than averaged away.

Seeds. Test shifts use seeds 5000 to 5029. No seed in that range is used anywhere in training or model selection: the look-ahead collection uses seeds from 12345 upwards, the selection of the best training round uses 1000 to 1003, the bound β was selected on 1000 to 1019, and PPO draws random 31-bit seeds during training. The development seeds overlap each other, which is harmless — they are all part of building the model — but none of them touches the test range.

Both separations are declared in the configuration file rather than in a script, so an evaluation cannot silently drift into its training data; the test seed range and the list of training scenarios are read from `params.yaml` by the training and evaluation tools alike.

6.3 Metrics

The primary metric is the objective of Equation (3.1), the shift score. It is reported together with the quantities it is composed of, because two policies can reach the same score for opposite reasons:

Delivered units

— what the factory gets: the throughput.

Lost production

— units not made because a buffer was full and its line stood BLOCKED. This is the cost of a decision not taken.

Energy

— watt-hours drawn from the battery over the shift, and watt-hours per delivered unit, which is the efficiency figure comparable across policies of different throughput.

Charging trips

and **minimum battery** — how a policy manages its energy budget: many short charges and a high floor, or few charges and a low one.

Unsafe shifts

— the number of shifts containing at least one safety violation.

Decision latency

— milliseconds per decision, excluding the first decision of a shift, which loads the model.

A *safety violation* is recorded when the battery falls below the reserve while the robot is away from the charger, when the motor temperature reaches its limit, or when the battery is empty. Repeated occurrences of the same kind in a row count once, so the count measures how often a policy entered an unsafe state, not how long it stayed there. The score charges 20 points per violation, which — as Section 7.1 notes — is still not enough to make an unsafe policy’s score comparable to a safe one’s; the two numbers are therefore always reported together.

6.4 Statistics

Means are reported with 95 % bootstrap confidence intervals (2000 resamples of the shifts of that policy and scenario). The bootstrap is used because shift scores are not normally distributed: in scenarios with random machine failures the distribution is skewed by shifts in which an early fault cost most of the hour.

Differences between policies are tested with the *paired Wilcoxon signed-rank test* on the matched seeds, at a significance level of 0.05. The test is paired for the reason given in Section 6.1, and non-parametric because it makes no assumption about the shape of the score distribution. The neuro-symbolic model is the reference in every comparison, so a reported p -value always answers the same question: *is this policy's score different from the neuro-symbolic model's, on the same shifts?*

Two honest caveats. First, no correction for multiple comparisons is applied: with eight policies compared against the reference across twelve scenarios, some individually significant results are expected by chance alone, which is why the discussion relies on differences that are both significant and consistent across scenarios rather than on isolated ones. Second, statistical significance is not importance: with thirty paired seeds, a difference of half a point can reach $p < 0.001$ while meaning little in a factory. Effect sizes are therefore always given in the same sentence as the p -value.

6.5 Validation in the full simulation

Every number in Sections 7.1, 7.4 and 7.5 comes from the fast simulator, which abstracts driving into distances, durations and energies. Whether those numbers describe the real system is a separate question, and it is answered by running complete shifts through the full stack — Gazebo physics, Nav2 planning and control, AMCL localisation, the real mission executor and the decision service — and comparing each of them against the fast simulator on the same policy, scenario and seed.

The procedure for one validation shift is:

1. Start the complete system headless with the chosen scenario, model, seed and shift length.
2. Wait for the mission summary the executor writes when the shift ends, then stop that run and confirm that no process from it survives — a leftover simulator would publish a second clock and corrupt the next run.
3. Compute the score from the run’s own logs, applying the definitions of Section 6.3: delivered units from the mission summary, lost production from the per-section logs, energy from the robot telemetry, and violations from the recorded battery and temperature traces.
4. Replay the same policy, scenario, seed and shift length in the fast simulator and compute the same quantities.

One detail decides whether the comparison is fair. The mission executor checks the clock only between actions, so an action that starts before the shift ends is always finished — a shift can therefore overrun, and in one case an action that began in the last minute was a seventeen-minute charge. The fast simulator normally stops at exactly the shift length, mid-action. For the comparison it is therefore run in a mode that also finishes the running action, so that both sides account for the same work. Without this, the fast simulator would appear to use systematically less energy, purely because it stopped earlier.

Twelve validation shifts of twenty minutes were driven (two models $\times$ three scenarios $\times$ two seeds), and two of sixty minutes on `low_battery_start`, because twenty minutes is too short for the battery to become the binding constraint. Wall-clock cost is the reason these are counted in tens and not thousands: a twenty-minute shift takes about 21 min of real time, and a sixty-minute shift a little over an hour.

6.6 The planner study

The planner comparison of Section 7.9 isolates the global planner: every planner computes the path, and the same local controller follows it, so only the route-finding differs. It was run at two levels. First on a fixed seven-leg tour of the hall, repeated five times per planner with the order of planners rotated between repeats, measuring planning time, path length against the shortest known path, turning and clearance. Then over complete twenty-minute shifts of the identical scenario and seed, five repeats per planner, alternating planners so that background load on the machine affects both equally — this alternation is what allows the faults reported in Section 7.9 to be attributed to the planner rather than to the computer.

6.7 Software and hardware

All experiments ran on one laptop: Intel Core i7-9750H (6 cores, 12 threads, 2.6 GHz), 23 GiB RAM. No GPU was used: training and inference run on the CPU throughout, which is itself a result worth noting: the models in this thesis are small enough that a factory would not need special hardware to run them.

The software stack is ROS 2 Jazzy with Nav2 (AMCL localisation, MPPI controller) and Gazebo Harmonic 8.11 running headless, Python 3.12, PyTorch 2.14 (CPU build) for the neural scorer, and Stable-Baselines3 2.9 with `sb3-contrib` for Maskable PPO.

Training costs, on that machine: the neural scorer takes about 40 min for three rounds of collection and training, of which roughly 33 min is look-ahead simulation; PPO takes 9.8 min for 300 000 steps, and the unmasked variant the same. The full evaluation of 3240 shifts takes 78 s. The Gazebo validation, by contrast, takes about 21 min per twenty-minute shift, which is the practical reason the experiment is structured in two tiers at all (Section 4.8).

Chapter 7

Results

This chapter reports what the policies of Chapter 5 actually did, under the protocol of Chapter 6. It is deliberately descriptive: the numbers are stated, the differences that are statistically supported are marked as such, and the interpretation is left to Chapter 8.

Unless stated otherwise, every figure in this chapter comes from the same experiment: nine policies, twelve scenarios, thirty seeds each, one simulated hour per shift — 3240 shifts in total, all policies on identical seeds. Confidence intervals are 95% bootstrap intervals of the mean score; p -values come from paired Wilcoxon signed-rank tests against the neuro-symbolic model on the same seeds. The complete tables, including every metric for every scenario, are in the result files listed in Appendix C.

One reading instruction matters throughout. Because a safety violation costs a fixed 20 points per *kind* of violation (Section 6.3) rather than per minute spent in the unsafe state, a policy that works through a violation can still finish a shift with a high score. Such a score is not a valid result: it is obtained by ignoring a constraint that the problem statement declares inviolable. Wherever a policy’s number includes shifts with violations, this is marked, and in Figure 7.1 those bars are hatched.

7.1 Overall comparison

Table 7.1 summarises all nine policies over the full set of scenarios.

Table 7.1: Results over all twelve scenarios (360 shifts per policy). Δ is the mean paired difference in score against the neuro-symbolic model; p is the paired Wilcoxon signed-rank test on identical seeds. “Unsafe shifts” counts shifts with at least one safety violation.

Policy	Score [95 % CI]	Delivered	Lost	W h	Unsafe shifts	Δ	p
NS (bounded)	53.81 [51.9, 55.7]	59.9	1.90	8.42	0/360	—	—
ns_unbounded	51.56 [49.5, 53.6]	58.2	2.56	8.08	0/360	−2.25	< 0.001
ns_estimate_only	52.85 [50.9, 54.8]	59.3	2.12	8.77	0/360	−0.96	< 0.001
ns_symbolic_only	52.91 [50.9, 54.9]	59.4	2.11	8.71	0/360	−0.90	< 0.001
ns_neural_only	51.81 [49.3, 54.2]	60.7	0.21	8.56	57/360	−2.00	0.109
rule	41.66 [38.5, 44.7]	51.6	7.47	4.97	0/360	−12.15	< 0.001
ppo	52.35 [50.3, 54.4]	61.9	0.13	10.21	53/360	−1.46	< 0.001
ppo_unmasked	28.19 [25.8, 30.5]	44.6	8.83	7.66	42/360	−25.62	< 0.001
ppo_wp6	53.75 [51.6, 55.8]	62.1	0.35	9.95	51/360	−0.06	0.121

Among the policies that never violated a constraint, the neuro-symbolic model achieves the highest score. Its advantage over the rule-based reference is large (12.15 points, $p < 0.001$); its advantage over its own symbolic layer is small but consistent (0.90 points, $p < 0.001$), and the same holds against the variant in which the network is switched off entirely (**ns_estimate_only**, 0.96 points, $p < 0.001$).

Two policies reach comparable scores by other means. **ppo** delivers more units than any rule-carrying policy (61.9 against 59.9) and loses almost no production, but spends 10.21 W h against 8.42 W h and violates a constraint in 53 of its 360 shifts. **ppo_wp6**, the earlier model trained on only four of the six training scenarios, is statistically indistinguishable from the neuro-symbolic model in score ($p = 0.121$) and equally unsafe (51/360). The interpretation of this pattern is taken up in Section 8.3.

Shift score per policy and scenario (mean, 95 % CI; higher is better; hatched = shifts with safety violations)

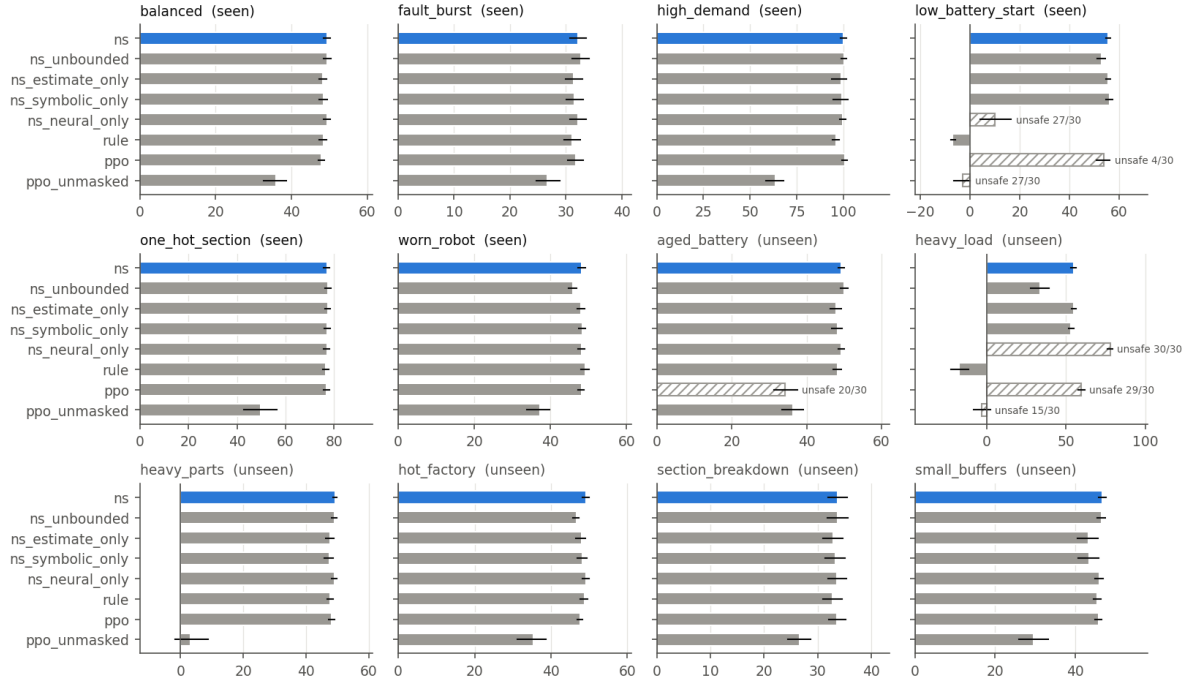


Figure 7.1: Score per policy and scenario (mean over 30 seeds, 95 % bootstrap CI). Hatched bars mark policies whose mean includes shifts with safety violations; the annotation gives how many of the 30 shifts were affected.

7.2 Per-scenario behaviour

The pooled numbers hide how differently the policies behave from one factory to the next. Table 7.2 gives the score of the four policies of primary interest in each scenario.

Table 7.2: Score per scenario (mean over 30 seeds). * marks a difference from the neuro-symbolic model that is significant at $p < 0.05$ (paired Wilcoxon). Entries marked *unsafe* contain shifts with safety violations and are therefore not comparable on score alone.

Scenario	Split	NS	ns_symbolic_only	ppo	rule
balanced	seen	49.36	48.42*	47.93*	48.37*
high_demand	seen	99.99	99.16	100.60	95.90*
one_hot_section	seen	77.11	77.31	77.03	76.66
fault_burst	seen	32.25	31.59	31.79	31.15*
low_battery_start	seen	55.57	56.08	53.76 (4 unsafe)	−6.97*
worn_robot	seen	48.24	48.37	48.15	49.17*
heavy_load	unseen	54.88	53.21*	59.47* (29 unsafe)	−17.51*
heavy_parts	unseen	49.23	47.44*	48.18*	47.69*
small_buffers	unseen	46.91	43.51*	45.85*	45.65*
section_breakdown	unseen	33.75	33.27	33.65	32.83*
hot_factory	unseen	49.17	48.20	47.58*	48.71
aged_battery	unseen	49.26	48.35	34.25* (20 unsafe)	48.25

The neuro-symbolic model has the highest score among the safe policies in eight of the twelve scenarios. It is beaten in `low_battery_start` and `one_hot_section` by its own symbolic layer and in `high_demand` by ppo, in all three cases by margins that are not statistically supported, and in `worn_robot` by the rule-based reference (0.93 points, $p = 0.033$) — the one case where a simpler policy wins significantly. The scenarios also differ enormously in difficulty: scores range from −17.51 (the rule policy under `heavy_load`) to 100.60 (ppo under `high_demand`), which is why pooled means are reported together with the per-scenario breakdown rather than instead of it.

7.3 What each half contributes

The ablations isolate the two halves of the model. Switching the network off but keeping the rules and the analytic estimate (`ns_estimate_only`) costs 0.96 points overall ($p < 0.001$); replacing the analytic estimate by the symbolic layer’s own cheapest-per-unit ranking (`ns_symbolic_only`) costs 0.90 points ($p < 0.001$). Removing the rules instead and letting the network decide alone (`ns_neural_only`) produces a score that is not

significantly different from the full model over all scenarios ($p = 0.109$) — but reaches it while violating a constraint in 57 of 360 shifts.

The most informative ablation is **ns_unbounded**: the same network, the same rules, the same training, with the only difference that its correction is not limited (Section 5.4). Over all scenarios it scores 2.25 points less than the bounded model ($p < 0.001$). Table 7.3 shows where that difference comes from.

Table 7.3: Seen against unseen scenarios (180 shifts each). The bound costs nothing on the scenarios the network was trained on and gains substantially on those it was not.

Policy	Seen (trained on)		Unseen	
	Score	Unsafe	Score	Unsafe
NS (bounded)	60.42	0/180	47.20	0/180
ns_unbounded	59.79 ($p = 0.058$)	0/180	43.33 ($p < 0.001$)	0/180
ns_symbolic_only	60.16 ($p = 0.857$)	0/180	45.66 ($p < 0.001$)	0/180
ns_estimate_only	59.92 ($p = 0.101$)	0/180	45.78 ($p < 0.001$)	0/180
ns_neural_only	52.77 ($p < 0.001$)	27/180	50.84 ($p = 0.034$)	30/180
rule	49.05	0/180	34.27	0/180
ppo	59.88 ($p = 0.213$)	4/180	44.83 ($p < 0.001$)	49/180
ppo_wp6	59.23 ($p = 0.313$)	11/180	48.28 ($p = 0.251$)	40/180

On the six scenarios the network was trained on, bounding its influence changes almost nothing (60.42 against 59.79, $p = 0.058$). On the six it never saw, the same bound is worth 3.87 points ($p < 0.001$). The bound therefore does not make the network better; it prevents it from becoming harmful where its training no longer applies.

7.4 Generalisation

Every policy scores lower on the unseen scenarios than on the seen ones, which is expected: those six scenarios were written to be harder. What differs is how much each policy loses and what it sacrifices.

The clearest case is **heavy_load**, a factory producing beyond what one robot can carry. Here the unbounded model scores 33.78 and the bounded one 54.88, a difference

of 21.11 points ($p < 0.001$) — by far the largest effect of the bound anywhere in the evaluation, and the observation that motivated it (Section 5.4). Both variants are safe; the difference is entirely lost production, 30.06 against 21.84 units.

Two policies score higher than the neuro-symbolic model under `heavy_load`: `ns_neural_only` with 77.73 and `ppo` with 59.47. Both do so by working the battery down to a minimum of 3.4% and 1.4% respectively, and both violate the reserve constraint in 30 and 29 of their 30 shifts. Under the rules of Section 3.5 these are not better solutions to the problem but solutions to a different, unconstrained one.

7.5 Safety

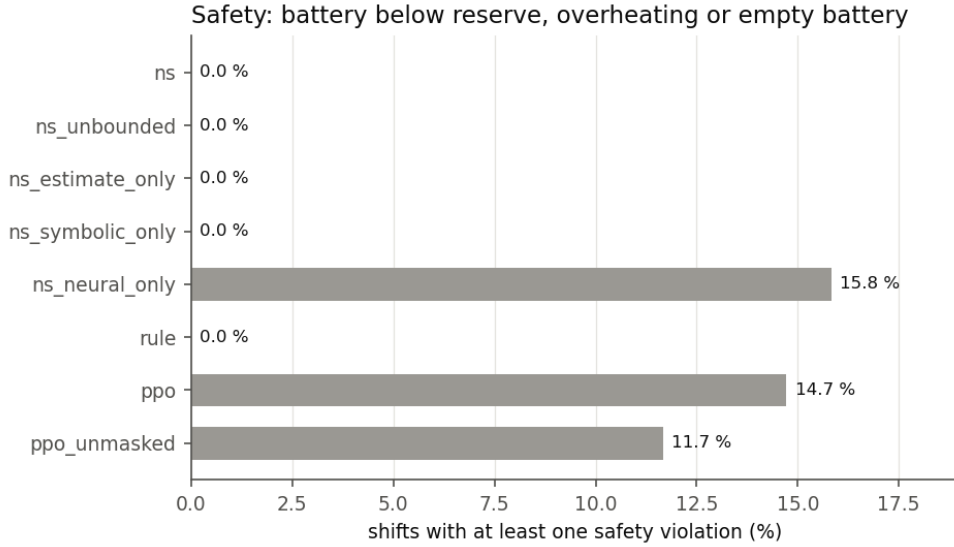


Figure 7.2: Share of shifts containing at least one safety violation (battery below the reserve away from the charger, motor temperature at the limit, or an empty battery).

The result is categorical rather than gradual. Every policy that carries the symbolic hard rules — the neuro-symbolic model, its bounded and unbounded variants, the estimate-only and symbolic-only ablations and the rule-based reference — completed all 1800 of their shifts without a single violation. Every policy without them violated constraints:

the same network without rules in 15.8 % of shifts, masked ppo in 14.7 %, unmasked ppo in 11.7 %.

For ppo the violations are concentrated where its training no longer covers the situation: 49 of its 53 unsafe shifts occur in unseen scenarios, and 29 of those in `heavy_load` alone. Retraining PPO on all six training scenarios instead of four did not change this (ppo_wp6, 51/360), which indicates that the violations follow from learning the constraint from a reward signal rather than from having seen too few scenarios.

7.6 Throughput and energy

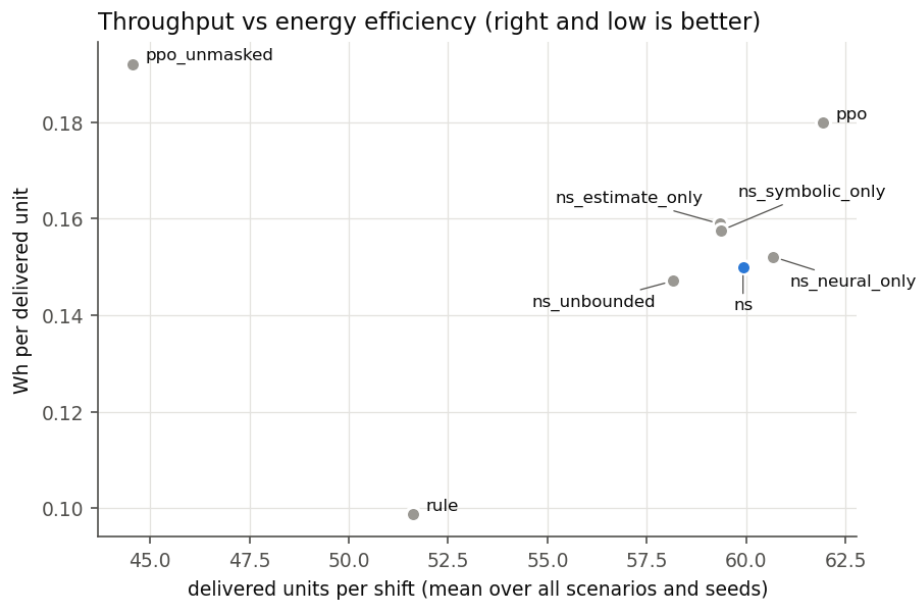


Figure 7.3: Delivered units against energy per delivered unit, averaged over all scenarios and seeds. Points towards the lower right deliver more for less energy.

The rule-based reference is by far the most energy-efficient policy (0.099 W h per delivered unit against 0.150 W h for the neuro-symbolic model), and it achieves this by charging often — 7.1 charging trips per shift against 2.8 — and therefore spending much of the shift parked, where idling is paid by the charger rather than by the battery. Its throughput

suffers accordingly: 51.6 delivered units against 59.9, and 7.47 lost units against 1.90. This accounting asymmetry is a known limitation of the objective and is discussed in Section 8.5.

`ppo` sits at the opposite end: it delivers the most units of any policy (61.9) and spends the most energy (10.21 W h, 0.180 W h per unit), with the lowest mean minimum battery of the safe-scoring policies (45.0 % against 59.0 % for the neuro-symbolic model). Its behaviour is consistent: drive more, charge less, stay closer to the limits.

7.7 Decision latency

Mean time per decision was 1.46 ms for the neuro-symbolic model, 1.39 ms for the symbolic layer alone, 2.67 ms for `ppo`, measured on the evaluation machine excluding the first decision of each shift, which loads the model. Since one action lasts between 30 s and several minutes, decision time is irrelevant to the result for every policy tested; it is reported only to establish that neither approach is disqualified by its computational cost.

7.8 Validation against the full simulation

All numbers so far come from the fast simulator. To establish what they are worth, twelve complete shifts were driven in Gazebo with the full navigation stack, and each was replayed in the fast simulator with the same policy, scenario and seed (Section 6.5).

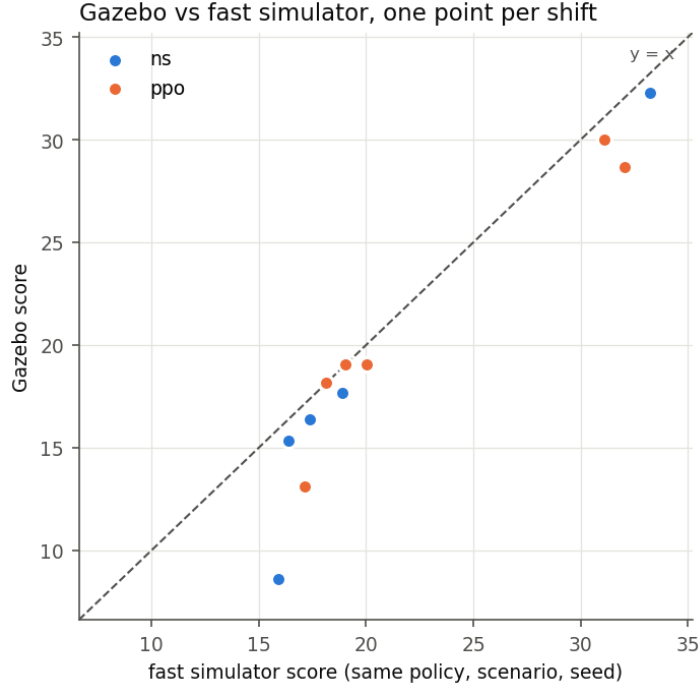


Figure 7.4: Gazebo score against fast-simulator score for the same policy, scenario and seed. A point on the dashed line would mean perfect agreement.

Over the eleven shifts that ran to completion, the fast simulator and Gazebo agree closely on energy and slightly less on throughput: mean energy 3.24 W h against 3.24 W h (0.2%), delivered units 23.7 against 25.0 (−5.1%), score 21.99 against 23.32 (−5.7%). The Gazebo runs are consistently the slightly weaker ones: real driving loses a few seconds per leg to acceleration, recovery behaviours and arrival tolerances, which over a twenty-minute shift costs about one delivered unit. No completed shift contained a safety violation on either side, and the ordering of the policies was the same in both.

Twenty-minute shifts are, however, too short for the battery to become critical, which is where the policies differ most. One full sixty-minute shift was therefore driven per model on `low_battery_start` (Table 7.4).

Table 7.4: Full sixty-minute shifts in Gazebo, scenario `low_battery_start`, seed 5000.

Model	Score	Delivered	W h	Minimum battery	Violations
NS	55.16	61	7.36	17.0 %	0
ppo	37.79	62	8.42	9.4 %	1

With almost identical throughput (61 against 62 units), **ppo** drove the battery below the 15 % reserve and continued to work: the mission log shows four consecutive actions completed at 14.9 %, 14.2 %, 13.5 % and 12.9 % away from the charger, with a minimum of 9.4 %. The neuro-symbolic model never fell below 17.0 %. The safety difference measured statistically in Section 7.5 therefore reproduces in the full simulation, with real navigation, real timing and real recovery behaviour.

7.9 Global path planner

The decision layer chooses destinations; a global planner chooses the route. Five Nav2 planners were compared, first on short tours and then over complete shifts, with the same controller in every case so that only the planner differs.

On the short benchmark — a fixed seven-leg tour, five repeats per planner — Theta* was the fastest, completing a tour in 147.8(9) s against 152.7(19) s for the NavFn Dijkstra planner ($p = 0.001$), with about half the total turning and the shortest planning times. On that evidence it was made the default.

Over full twenty-minute shifts the picture reversed. In five repetitions of an identical shift per planner, three of the five Theta* shifts contained navigation faults — six planner-loop stalls, three refusals to plan and six navigation retries — while all five NavFn shifts were clean. The refusals carry their own explanation in the planner’s message: **Either of the start or goal pose are an obstacle!**. Theta* refuses to plan when the start or goal cell is momentarily occupied in the costmap, which happens when the robot stands close to a machine; NavFn plans from the nearest free cell within its tolerance of 0.5 m instead. Counting every Gazebo shift driven in this work, Theta* produced faults in 6 of

17 shifts and NavFn in 0 of 5. The speed advantage did not survive either: over whole shifts the two planners scored 18.07 (sd 1.61) and 17.75 (sd 1.23), a difference well inside the run-to-run spread. NavFn Dijkstra was therefore restored as the default.

7.10 Summary

- Among policies that never violated a constraint, the neuro-symbolic model scored highest overall (53.81), ahead of its symbolic layer alone (52.91, $p < 0.001$) and the rule-based reference (41.66, $p < 0.001$).
- Bounding the neural correction is worth almost nothing on the training scenarios ($p = 0.058$) and 3.87 points on unseen ones ($p < 0.001$), with the largest single effect in `heavy_load` (54.88 against 33.78).
- Every policy carrying the symbolic rules completed 1800 shifts without a violation; every policy without them violated constraints in 11.7% to 15.8% of shifts, mostly in scenarios outside their training.
- `ppo` matches the neuro-symbolic model on score in seen scenarios ($p = 0.213$) by delivering more and spending 21% more energy, and is unsafe in 49 of its 180 unseen shifts.
- The fast simulator agrees with the full Gazebo simulation to 0.2% on energy and 5.7% on score, and the safety difference between the neuro-symbolic model and `ppo` reproduces in a full-length Gazebo shift.
- A planner selected on short tours proved unreliable over full shifts, and was replaced.

Chapter 8

Discussion

Chapter 7 reported what happened. This chapter asks what it means: what the numbers say about the research questions, why the central mechanism works, which measurement decisions changed the conclusions, and where the work’s limits are.

8.1 Answering the research questions

RQ1 — Does the neuro-symbolic model outperform deep reinforcement learning of equal information? On score alone, the honest answer is *barely*: 53.81 against 52.35 over all scenarios, and on the six training scenarios the difference is not statistically supported ($p = 0.213$). On the terms the problem actually sets, the answer is *yes, and not narrowly*: the neuro-symbolic model never violated a constraint in 360 shifts, while `ppo` did so in 53, and it reached its throughput with 21 % less energy (8.42 W h against 10.21 W h) and a mean minimum battery of 59.0 % against 45.0 %. Both policies solve a problem well; only one of them solves the problem as stated in Section 3.5.

RQ2 — What does each half contribute? The rules contribute the guarantee, and they are not merely a safety net: the symbolic layer alone scores 52.91, above `ppo` and far above the rule-based reference. The network contributes a small, consistent improvement on top — 0.90 points against the symbolic layer and 0.96 against the analytic estimate

alone, both at $p < 0.001$. Neither half is redundant, but they are not equal partners: the rules carry most of the competence, and the network refines it. The most striking evidence for the division of labour is `ns_neural_only`. Without rules, the same network delivers *more* units than the full model (60.7 against 59.9) and loses almost no production (0.21 against 1.90) — it has genuinely learned what is worth doing — while violating constraints in 57 of 360 shifts. It learned the judgement and not the guarantee, which is precisely the split the architecture assumes.

RQ3 — Do the policies hold up in situations they were never trained for?

This is where the approaches separate most clearly. On the six unseen scenarios the neuro-symbolic model scores 47.20, its unbounded variant 43.33, the symbolic layer 45.66 and `ppo` 44.83 with 49 unsafe shifts out of 180. Two observations follow. First, an unbounded learned ranker is worse than no ranker at all outside its training distribution: the network does not merely stop helping, it actively harms. Second, PPO’s failures are concentrated where its training no longer covers the situation — 49 of its 53 unsafe shifts are in unseen scenarios, 29 of them in `heavy_load` alone.

RQ4 — Do fast-simulator results transfer to the full simulation? Yes, with a known bias. Energy transfers almost exactly (0.2%) because it is computed from the same physical model in both cases; throughput is 5.1% lower in Gazebo because real driving loses seconds per leg to acceleration, arrival tolerances and occasional recovery behaviours. The bias is systematic and in one direction, so it shifts all policies together and leaves their ordering intact — which is what a comparison needs. The sixty-minute shifts additionally reproduced the safety difference itself (Table 7.4), so the central claim of this thesis does not rest on the abstraction.

8.2 Why bounding the correction works

The bound of Equation (5.2) is one line of code, and it is worth being precise about why such a small change is worth 3.87 points on unseen scenarios.

The score of an action is the sum of two terms with different epistemic status. The analytic estimate is arithmetic on the current state: given the distances, the payload and the objective weights, it computes what an action moves and what it costs. Arithmetic *extrapolates* — it is as valid in a factory producing twice as fast as in the one it was written for. The learned correction is a function fitted to a sample of states; it *interpolates*. Inside the region its training covered, it encodes real knowledge that the arithmetic lacks: which line will block next, when a fuller load repays the detour. Outside that region, it continues to produce confident numbers with no basis.

Unbounded, the second term dominates the first: the corrections observed in the failing scenario were around +20 on every candidate, while the estimates differed by a few units. The ranking was therefore decided entirely by the network, in a factory where its training did not apply, and the rules could not intervene because none of its choices were illegal — waiting with cargo while lines block is wasteful, not forbidden.

Bounding the correction re-establishes the intended hierarchy without touching either component. Where the arithmetic is nearly indifferent, the network decides, and its judgement is exactly what is wanted. Where the arithmetic is clear, no amount of misplaced confidence can overturn it. The selection table (Table 5.1) shows both halves of this: $\beta = 0$, which silences the network, is worse than every bounded setting, and so is $\beta = \infty$.

This is a different mechanism from the shielding and action masking discussed in Section 2.4. Those act on the *legality* of actions: a shield removes what is unsafe, and the learned policy is free within what remains. The bound acts on the *influence* of the learned component over choices that are all legal. The two are complementary — this system uses both — and the distinction is worth stating, because a shield would not have prevented the failure described above.

8.3 Why the rules matter more than the score suggests

A reader comparing only scores would conclude that the difference between the approaches is minor. The comparison of `ppo_wp6` with the neuro-symbolic model makes the point sharply: the two are statistically indistinguishable in score ($p = 0.121$), and one of them enters an unsafe state in 51 of 360 shifts.

What a violation means physically is not a slightly lower score. The battery reserve exists so the robot can always reach the charging station; below it, the robot risks stopping in a corridor, where it blocks a route, requires a person to fetch it, and delivers nothing further that shift. The scoring penalty of 20 points is a modelling convenience, not a real cost — the real cost is an unplanned human intervention in a factory that was supposed to run by itself. This is why Chapter 7 reports unsafe shifts beside every score rather than trusting the penalty to express the difference.

There is a second, quieter argument for the rules. The neuro-symbolic model explains every decision in terms a plant engineer can check: which constraint forbade an alternative, which priority applied, by how much the network preferred the chosen action. When the robot behaved oddly in Section 5.2.1, that explanation is what made the cause findable in minutes. A policy whose output is an action index affords no such inspection — a property that matters more in an industrial setting than a fraction of a point of score.

8.4 Measurement decisions that changed the conclusions

Two results in this thesis exist only because a measurement was questioned, and both changed a decision that had already been made.

An unfair training setup. PPO’s parallel environments were originally each bound to one scenario, so with four environments the agent saw four of the six training scenarios.

Had this gone unnoticed, the thesis would have compared a fully trained neuro-symbolic model against a partially trained baseline, and the comparison would have been worthless regardless of its statistics. The correction is undramatic — after retraining on all six, PPO’s score and its violation rate are essentially unchanged — but that outcome is itself the useful finding: PPO’s constraint violations are not a symptom of insufficient scenario coverage, and would not have been fixed by more of the same training.

A benchmark that measured the wrong regime. The global planner was selected on a short benchmark tour, where Theta* was 3.3% faster with half the turning — a clean, statistically significant result ($p = 0.001$). Over full shifts the same planner produced faults in 6 of 17 shifts, including one that ended with the robot halted, because it refuses to plan when the start or goal cell is momentarily occupied. The benchmark was not wrong; it answered the question it was asked, which was about tours between free poses in an empty hall. The operating regime — a robot standing against a machine, in a costmap containing fresh sensor data — was outside it. The lesson generalises beyond planners: a benchmark measures a regime, and a component chosen in one regime has to be re-checked in the one it will run in.

8.5 Limitations

The objective does not charge for charging. Only energy drawn from the battery is counted (Section 3.4). A robot parked on the charger draws its idle power from the charging station, so waiting there is free in the score although the factory pays for it. This favours policies that spend time docked, and it is the main reason the rule-based reference appears so energy-efficient (0.099 Wh per unit against 0.150 Wh): it charges 7.1 times per shift and spends much of the hour parked. Correcting this would require re-training both learned models against the corrected objective, so it is reported rather than fixed. The comparison between policies remains valid — all of them are scored the same way — but the absolute energy figures understate what a real installation would

pay.

The remaining weakness of the model is a batching trade-off. In `worn_robot`, the only scenario where a simpler policy wins significantly, the rule reference makes 14 deliveries of 3.7 units and drives 359m; the neuro-symbolic model makes 25 deliveries of 2.2 units and drives 468m for the same output. Reacting quickly is what wins everywhere else, and it is what loses here, because a worn drivetrain makes every metre more expensive. The model has no notion that waiting for a fuller load can be the cheaper option; its look-ahead horizon of 900s may simply be too short for that trade-off to appear in the training targets.

One robot, one hall, one objective. The problem has a single robot, a single delivery point and a single charger, and no human traffic. Several robots would introduce congestion, charger contention and task allocation — a different problem class in which the symbolic layer’s per-action reasoning would have to be extended considerably. Likewise, all results come from one maze layout and one set of objective weights; the ratio between the cost of a lost unit and the cost of a watt-hour in particular is a modelling choice, and a factory that valued energy differently might rank the policies differently.

The symbolic layer is hand-written. Its rules are only as good as the understanding of whoever wrote them, and H5 is direct evidence: a necessary rule was missing until a real shift exposed it. There is no mechanism guaranteeing that no further such gap exists. Learning the symbolic layer, as some of the related work does (Section 2.3), would address this, at the cost of the readability the hand-written rules provide.

Simulation throughout. No physical robot was involved. The energy model is calibrated and validated against Gazebo, but Gazebo shares its assumptions about rolling resistance and motor efficiency, so agreement between them is consistency rather than proof. Localisation error, wheel slip, floor variation and battery ageing would all test the model in ways this work cannot.

The full-simulation evidence is thin in volume. Twelve twenty-minute shifts and two sixty-minute shifts are demonstrations, not statistics. They establish that the fast simulator does not mislead and that the safety difference is real in the full stack; they cannot support a confidence interval on the sim-to-real gap.

8.6 Threats to validity

Internal validity. Policies are compared on identical seeds with paired non-parametric tests, which removes the dominant source of variance. No correction for multiple comparisons is applied (Section 6.4), so isolated significant results should be treated with caution; the conclusions drawn here rest on differences that are consistent across scenarios or supported by a mechanism, not on single p -values. The bound β was selected on training scenarios and development seeds only, so the improvement it produces on the test set is not a selection effect.

External validity. The results describe this factory: one layout, three sections with these rates, this robot, this objective. The scenario set was deliberately widened to twelve configurations, six of them never seen during training, which supports claims about robustness to *parameter* changes — rates, masses, buffer sizes, battery health, ambient temperature. It supports no claim about a different hall geometry, more robots, or a different task structure.

Construct validity. Whether the score in Equation (3.1) expresses what a factory wants is a modelling assumption. Two of its weights are defensible from first principles (a delivered unit and a lost unit are worth the same; energy is cheap relative to production), while the penalty of 20 per safety violation is a stand-in for a cost — an unplanned intervention — that has no natural unit. This thesis therefore avoids resting any conclusion on the penalty’s magnitude, and reports violation counts separately throughout.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

9.2 Future work

Bibliography

Appendix A

Configuration Parameters

Appendix B

Scenarios

Appendix C

Reproducing the Results